

# AWS CI/CD Project – Overview and Architecture

## Objective

The goal of this project is to build a **CI/CD pipeline on AWS** to fetch application code, build it, and deploy it on **AWS Elastic Beanstalk**, using AWS-native services.

---

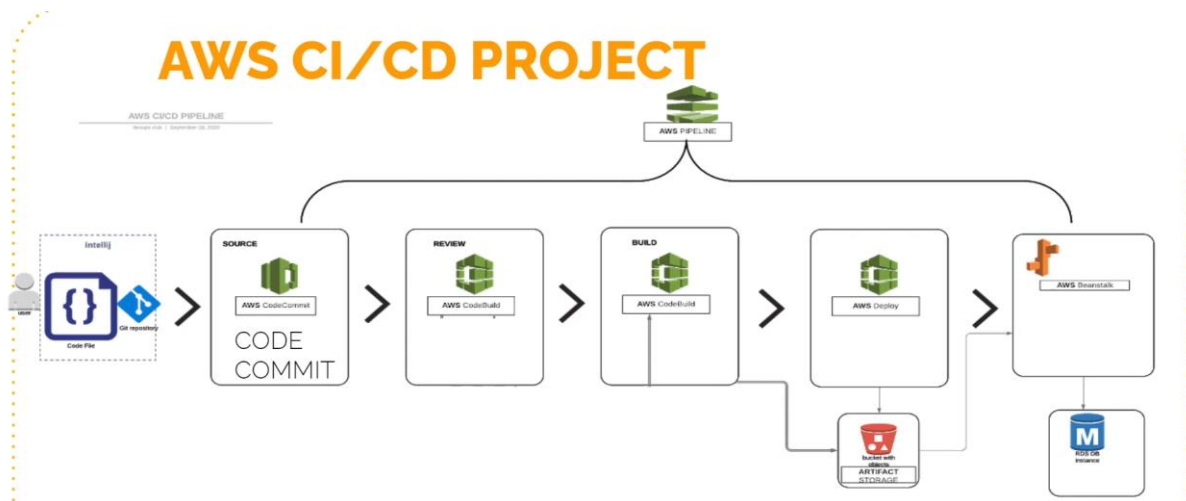
## Key Components of the Project

1. **AWS Elastic Beanstalk**
    - Hosts and deploys the application.
    - CodeBuild artifacts will be deployed here.
  2. **Amazon RDS (Relational Database Service)**
    - Provides the database backend for the **Vprofile application**.
    - Stores schemas and tables required by the app.
  3. **Source Code Repository – Bitbucket**
    - Vprofile source code originally resides on **GitHub**.
    - Code will be **migrated from GitHub to Bitbucket**.
    - Bitbucket works similarly to GitHub, offering Git-based repositories and integrated CI/CD capabilities.
  4. **AWS CodeBuild**
    - Acts like the "build" step in Jenkins pipelines.
    - Fetches source code from Bitbucket.
    - Builds the source into deployable artifacts.
  5. **AWS CodePipeline**
    - Orchestrates the CI/CD process.
    - Detects commits in the Bitbucket repo.
    - Triggers CodeBuild to compile and package the code.
    - Deploys the artifacts to **Elastic Beanstalk**.
- 

## CI/CD Workflow (High-Level)

1. Developer commits code to **Bitbucket repository**.
  2. **CodePipeline** detects the commit and triggers execution.
  3. **CodeBuild** fetches the source code → builds it → creates artifacts.
  4. CodePipeline deploys artifacts onto **Elastic Beanstalk environment**.
  5. Application connects to **Amazon RDS** for database functionality.
- 

## Architectural Diagram (Explained)



- **Source:** Bitbucket Git repository.
- **Pipeline Orchestration:** AWS CodePipeline.
- **Build:** AWS CodeBuild.
- **Deployment:** AWS Elastic Beanstalk.
- **Database:** Amazon RDS.

## 1.AWS Beanstalk Setup

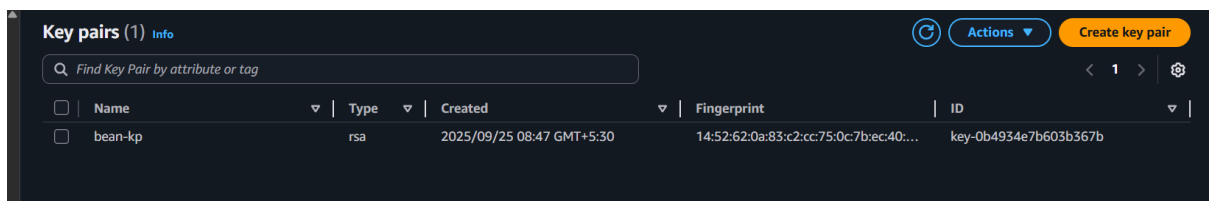
### Objective

Set up an **AWS Elastic Beanstalk environment** to deploy and run the Vprofile application.

### Steps to Set Up Beanstalk

#### 1. Create Key Pair

- Go to **EC2** → **Key Pairs**.
- Create a new key pair in **PEM format**.
- Example name: `bean-kp`.



- This will be used to log in to Beanstalk EC2 instances.

## 2. Create Beanstalk Application

- Open **Elastic Beanstalk** → **Create Application**.
- Choose **Web Server Environment**.

| Environment information                                                                                                             |                                               |
|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <b>Environment tier</b><br>Web server environment                                                                                   | <b>Application name</b><br>vprofile-bean-app  |
| <b>Environment name</b><br>Vprofile-bean-app-env                                                                                    | <b>Application code</b><br>Sample application |
| <b>Platform</b><br>arn:aws:elasticbeanstalk:us-east-1::platform/Tomcat 10 with Corretto 17 running on 64bit Amazon Linux 2023/5.7.5 |                                               |

- **Application Name:** vprofile-bean-app.
- **Environment Name:** Vprofile-bean-app-env (unique).
- **Domain Name:** Same as environment name (check availability).
- **Platform:** Tomcat 10 with Corretto 21 (Amazon Linux 2023).

## 3. Configure Environment

- **Configuration Type:** Custom.
- **Load Balanced Environment:** Multiple instances with load balancer.
- **Service Role:** Use an existing service role created earlier.
- **Instance Profile:** Select existing role (e.g., vprofile-bean-role).

**Configure service access** info

**Service access**  
IAM roles, assumed by Elastic Beanstalk as a service role, and EC2 instance profiles allow Elastic Beanstalk to create and manage your environment. Both the IAM role and instance profile must be attached to IAM managed policies that contain the required permissions. [Learn more](#)

**Service role**  
Choose an IAM role for Elastic Beanstalk to assume as a service role. The IAM role must have the required IAM managed policies.

aws-elasticbeanstalk-service-role ⌂ 🔄 Create role

**EC2 instance profile**  
Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.

vprofile-rearch-bean-role ⌂ 🔄 Create role

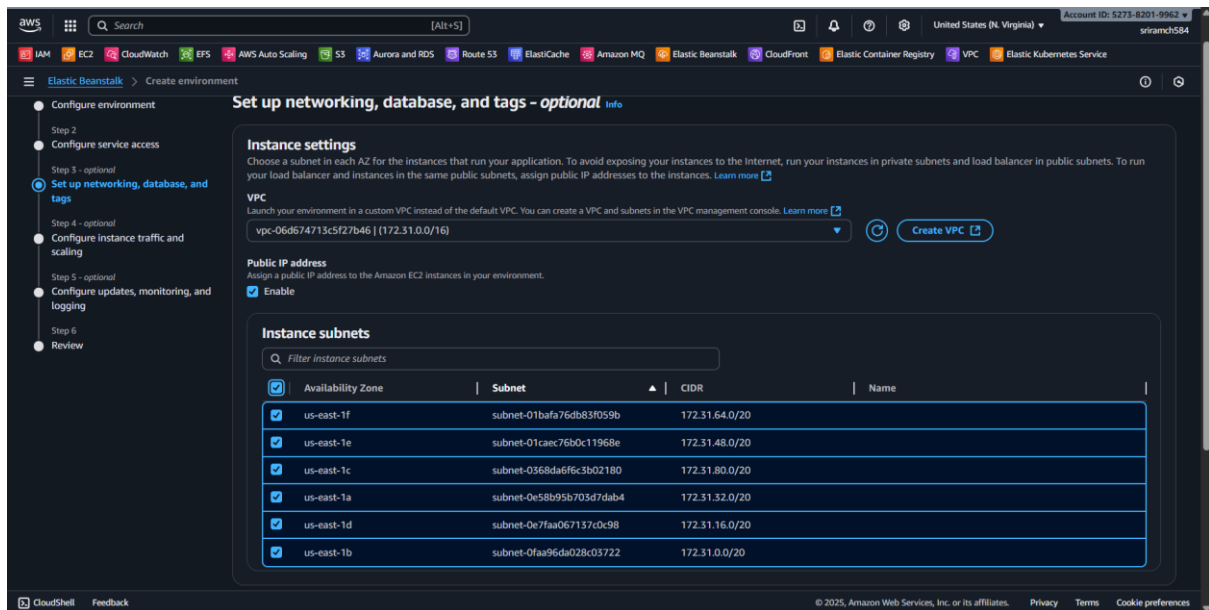
**EC2 key pair - optional**  
Select an EC2 key pair to securely log in to your EC2 instances. [Learn more](#)

bean-kp ⌂ 🔄

- **Key Pair:** Select the newly created key pair.

## 4. Networking

- Select **VPC** and enable **Public IP address**.



- Choose all subnets for multi-AZ availability.
- Minimum 2 instances across 2 availability zones.

## 5. Storage

- **Root Volume:** Change from *Container default* → **GP3**.

**Instances**
Info

Configure the Amazon EC2 instances that run your application.

Root volume (boot device)

**Root volume type**  
The type of storage for the root volume attached to each instance.

**Size**  
The number of gigabytes of the root volume attached to each instance.

**IOPS**  
Input/output operations per second for a provisioned IOPS (SSD) volume.

**Throughput**  
The desired throughput for the Amazon EBS root volume attached to your environment's EC2 instance

General Purpose 3(SSD)

10

3000

125

GB

IOPS

MIB/s

- **Reason:** GP3 enforces the use of **Launch Templates**, which prevents outdated **Launch Configurations**.

## 6. Auto Scaling and Instance Type

- **Auto Scaling Group:** Load balanced.
- **Min Instances:** 2.
- **Max Instances:** 2 (or 4, but scaling isn't required for this project).

**Environment type**  
Select a single-instance or load-balanced environment. You can develop and test an application in a single-instance environment to save costs and then upgrade to a load-balanced environment when the application is ready for production. [Learn more](#)

Load balanced

**Min instances**  
2 instances

**Max instances**  
4 instances

**Fleet composition**  
Spot instances are launched at the lowest available price. [Learn more](#)

☒ On-Demand instances  
☐ Combine purchase options and instances

**Architecture**  
The processor architecture determines the instance types that are made available. You can't change this selection after you create the environment. [Learn more](#)

☒ x86\_64  
This architecture uses x86 processors and is compatible with most third-party tools and libraries.

☐ arm64  
This architecture uses AWS Graviton processors. You might have to recompile some third-party tools and libraries.

**Instance types**  
Add instance types for your environment with your preferred launch order. The order preference only applies to On-Demand Instances and Spot Instances that use the capacity optimized prioritized allocation strategy. We recommend you include at least two instance types. [Learn more](#)

1. t2.micro

- **Instance Type:** t2.micro (eligible for free tier).

## 7. Load Balancer

- **Type:** Application Load Balancer (ALB).

**Load balancer network settings**

**Visibility**  
Make your load balancer internal if your application serves requests only from connected VPCs. Public load balancers serve requests from the Internet.

Public

**Dualstack (IPv4 & IPv6)**  
Enable IPv6 support for your load balancer. This requires a VPC and subnets with IPv6 enabled. IPv6 is not available because the selected VPC does not have IPv6 enabled.

☐ Enable

**Load balancer subnets**

Filter load balancer subnets

| <input checked="" type="checkbox"/> | Availability Zone | Subnet                   | CIDR           | Name |
|-------------------------------------|-------------------|--------------------------|----------------|------|
| <input checked="" type="checkbox"/> | us-east-1f        | subnet-01bafa76db83f059b | 172.31.64.0/20 |      |
| <input checked="" type="checkbox"/> | us-east-1e        | subnet-01caec76b0c11968e | 172.31.48.0/20 |      |
| <input checked="" type="checkbox"/> | us-east-1c        | subnet-0368da6f6c3b02180 | 172.31.80.0/20 |      |
| <input checked="" type="checkbox"/> | us-east-1a        | subnet-0e58b95b703d7dab4 | 172.31.32.0/20 |      |
| <input checked="" type="checkbox"/> | us-east-1d        | subnet-0e7faa067137c0c98 | 172.31.16.0/20 |      |
| <input checked="" type="checkbox"/> | us-east-1b        | subnet-0faa96da028c03722 | 172.31.0.0/20  |      |

- **Visibility:** Public.
- **Stickiness:** Enable session stickiness.

## Add process

Name

default

Port

80

Protocol

HTTP

▶ Health check

▼ Sessions

The following settings let you control whether the load balancer routes requests for the same session to the Amazon EC2 instance with the smallest load, or consistently to the same instance.

Session stickiness

☒ Enable

Cookie duration

Lifetime of the sticky session cookie between an Amazon EC2 instance and the load balancer.

86400

Cancel

Save

- Ensures users stay connected to the same instance.
- Important because **Vprofile app** does not use token-based authentication.

---

## 8. Deployment Settings

- **Health Reporting:** Default.
- **Rolling Updates:** Enable rolling deployment.

▼ **Rolling updates and deployments** [Info](#)

**Application deployments**

Choose how Amazon Elastic Beanstalk propagates source code changes and software configuration updates. [Learn more](#)

**Deployment policy**

Rolling

**Batch size type**

☒ Percentage

☐ Fixed

**Deployment batch size**

50

% instances at a time

**Traffic Split**

% to new application versions

**Traffic splitting evaluation time**

- **Deployment Size:** 50% (1 instance at a time).
  - For production, recommended size = **10–25%**.

## 9. Review and launch

- Review all configuration settings.
- Click **Submit**.
- Elastic Beanstalk will provision resources:
  - EC2 instances
  - Application Load Balancer
  - Auto Scaling Group
  - Related networking components.

The screenshot shows the AWS Elastic Beanstalk console. The top navigation bar includes the AWS logo, a search bar, and various service icons. The main content area displays the 'Vprofile-bean-app-env' environment overview. A blue banner at the top indicates that Elastic Beanstalk is launching the environment. The overview section includes fields for Health (Unknown), Environment ID (e-bcxmifkfh), Domain (vprofile-bean-app-1224.us-east-1.elasticbeanstalk.com), and Application name (vprofile-bean-app). The Platform section shows Tomcat 10 with Corretto 17 running on 64bit Amazon Linux 2023/5.7.5, with a 'Supported' platform state. The Events section shows a recent event from September 25, 2025, at 11:20:49 UTC+5:30, with the type 'INFO' and details about using an S3 storage bucket for environment data.

# 2.RDS & Application Setup on Beanstalk

# Objective

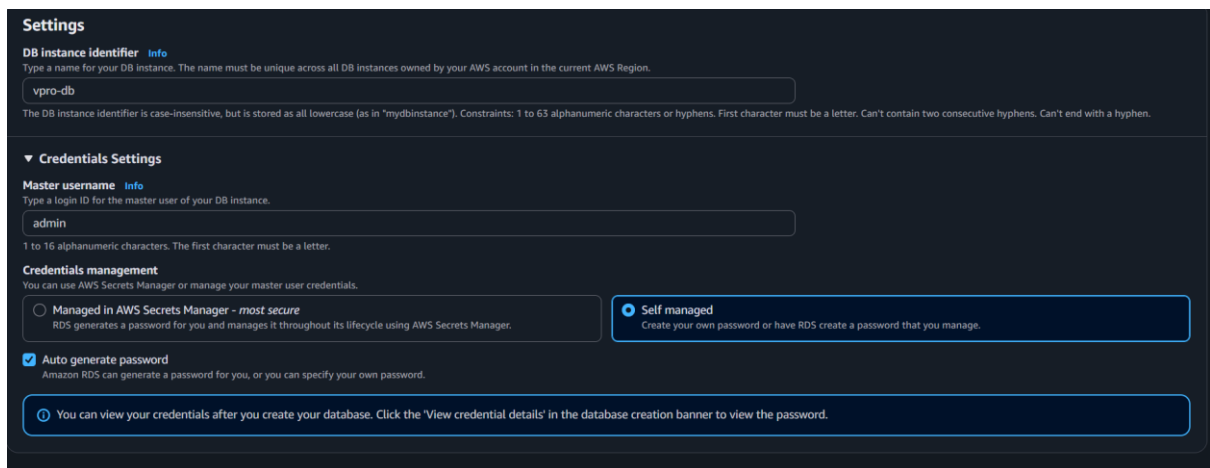
Set up **Amazon RDS** as the database backend for the Vprofile application and connect it with the **Elastic Beanstalk environment**. Initialize the database with required schemas and tables.

---

## Steps to Create Amazon RDS

### 1. Create RDS Instance

- Open **RDS service** → **Create Database**.
- **Creation Method**: Standard create.
- **Engine**: MySQL 8.0.35.
- **Template**: Free tier.
- **DB Instance Identifier**: `vprords`.
- **Master Username**: `admin`.



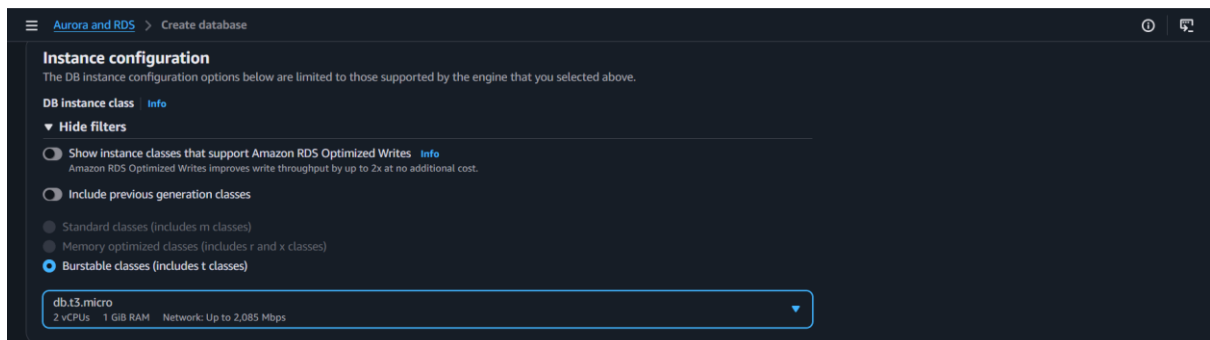
The screenshot shows the 'Settings' section of the Amazon RDS console. It includes fields for 'DB instance identifier' (set to 'vpro-db') and 'Master username' (set to 'admin'). Under 'Credentials management', the 'Self managed' option is selected. A note at the bottom states: 'You can view your credentials after you create your database. Click the 'View credential details' in the database creation banner to view the password.'

- **Password**: Auto-generate (make sure to copy and save).

---

### 2. Instance Configuration

- **Instance Type**: `t3.micro` or `t2.micro` (free tier).



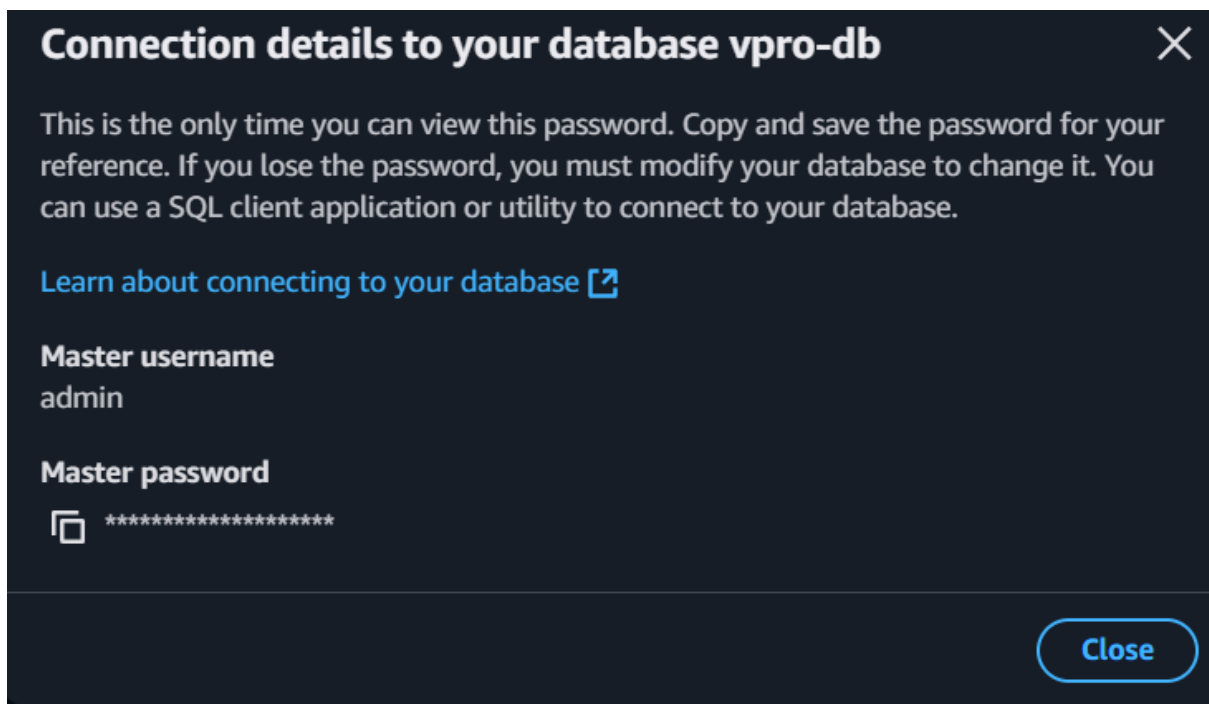
The screenshot shows the 'Instance configuration' section of the Amazon RDS console. It includes a 'DB instance class' dropdown menu set to 'db.t3.micro'. Below the dropdown, the specifications are listed: '2 vCPUs', '1 GiB RAM', and 'Network: Up to 2,085 Mbps'.



- **Storage:** General Purpose SSD (20 GB).
  - **Connectivity:**
    - Public Access: **No**.
    - Security Group: Create a new one (e.g., `vprords-sg`).
    - Port: Default **3306**.
  - **Database Name:** `accounts` (required by Vprofile app).
  - **Authentication:** Password authentication.
- 

### 3. Save Credentials

- Copy the auto-generated **username** and **password**.
- Store securely (needed in build configuration).



- If lost, you must reset master credentials later.
- 

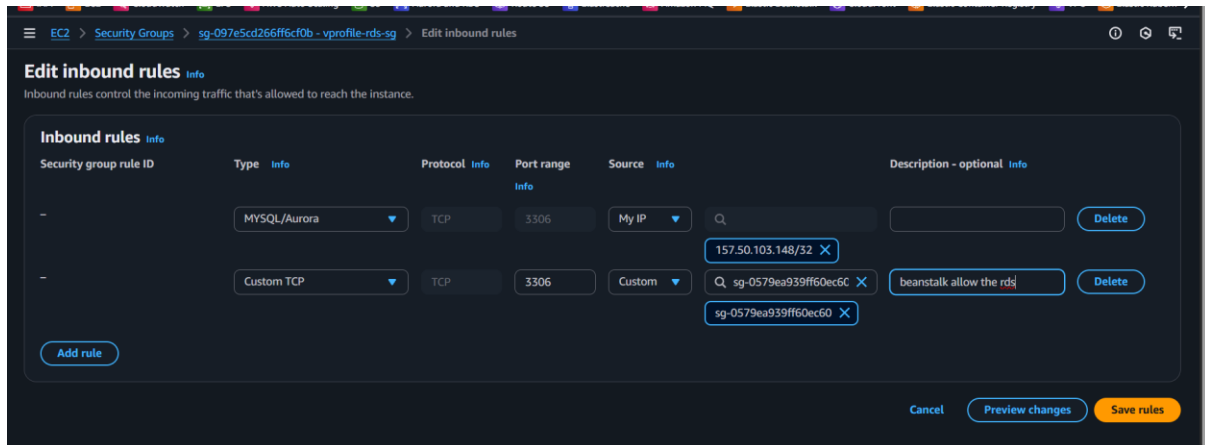
### 4. Wait for RDS Availability

- Takes 5–10 minutes.
  - Status should be **Available** before proceeding.
- 

## Connecting Beanstalk Instances to RDS

## 1. Update Security Group

- Go to RDS **security group** (vpro-rds-sg).
- Add inbound rule:
  - Type: MySQL/Aurora (3306).
  - Source: Beanstalk **instance security group**.



- This allows EC2 instances from Beanstalk to connect to RDS.

---

## 2. Verify Connection

1. SSH into any Beanstalk EC2 instance using key pair:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~ (main)
$ chmod 400 Downloads/bean-kp.pem
```



```

root@ip-172-31-62-143:~
[root@ip-172-31-62-143 ~]# mysql -h vpro-db.cixiwu6w0c1r.us-east-1.rds.amazonaws.com -u admin -p accounts
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 27
Server version: 8.0.42 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [accounts]> show tables

```

- Enter password when prompted.
- Use `SHOW DATABASES;` to confirm connection.

## Database Initialization

### 1. Download SQL File

- From Vprofile repo → Branch: AWS-CI.
- Path: `src/main/resources/db_backup.sql`.
- On EC2 instance, download file:
- `wget <RAW-FILE-URL>`

```

-- MySQL dump 10.13  Distrib 5.7.18, for Linux (x86_64)
--
-- Host: localhost    Database: accounts
--
-- Server version
--      5.7.18-0ubuntu0.16.10.1
--
/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
--
-- Table structure for table 'role'
--
DROP TABLE IF EXISTS `role`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE `role` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8;
/*!40101 SET character_set_client = @saved_cs_client */;
--
-- Dumping data for table 'role'
--
LOCK TABLES `role` WRITE;
/*!40000 ALTER TABLE `role` DISABLE KEYS */;
INSERT INTO `role` VALUES (1,'ROLE_USER');
/*!40000 ALTER TABLE `role` ENABLE KEYS */;
UNLOCK TABLES;
--
-- Table structure for table 'user'
--
DROP TABLE IF EXISTS `user`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE `user` (
  `id` int(11) NOT NULL AUTO_INCREMENT,

```

- Alternatively, copy contents manually into a file.

### 2. Run SQL Script

- Execute schema and table creation:

```

root@ip-172-31-62-143:~
[root@ip-172-31-62-143 ~]# mysql -h vpro-db.cixiwu6w0c1r.us-east-1.rds.amazonaws.com -u admin -p accounts < db_backup.sql
Enter password:

```

### 3. Verify Tables

- Log back into RDS:

```
[root@ip-172-31-62-143 ~]# mysql -h vpro-db.cixiwu6w0c1r.us-east-1.rds.amazonaws.com -u admin -p accounts
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 30
Server version: 8.0.42 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [accounts]> show tables;
+-----+
| Tables_in_accounts |
+-----+
| role                |
| user                |
| user_role           |
+-----+
3 rows in set (0.002 sec)

MySQL [accounts]> |
```

- Confirm required tables exist.

## 3. Bitbucket Repository Setup & Code Migration

### Objective

Create a Bitbucket repository, set up SSH-based authentication, and migrate the Vprofile application source code from GitHub to Bitbucket.

---

### Steps

#### 1. Create Bitbucket Account & Workspace

1. Go to [bitbucket.org](https://bitbucket.org).
2. Sign up using Google (or create a new account).
3. Create a **workspace** (e.g., aws-cicd-1000).
  - Workspace must be unique.
  - This will hold your repositories.

# Create a new repository

Import repository

Workspace  aws-cicd-1000

Project\*  vprofile  

Repository name\* vprofile-pro

Access level ☒ Private repository

Uncheck to make this repository public. Public repositories typically contain open-source code and can be viewed by anyone.

Include a README? No 

Default branch name e.g., 'main'

Include .gitignore? No 

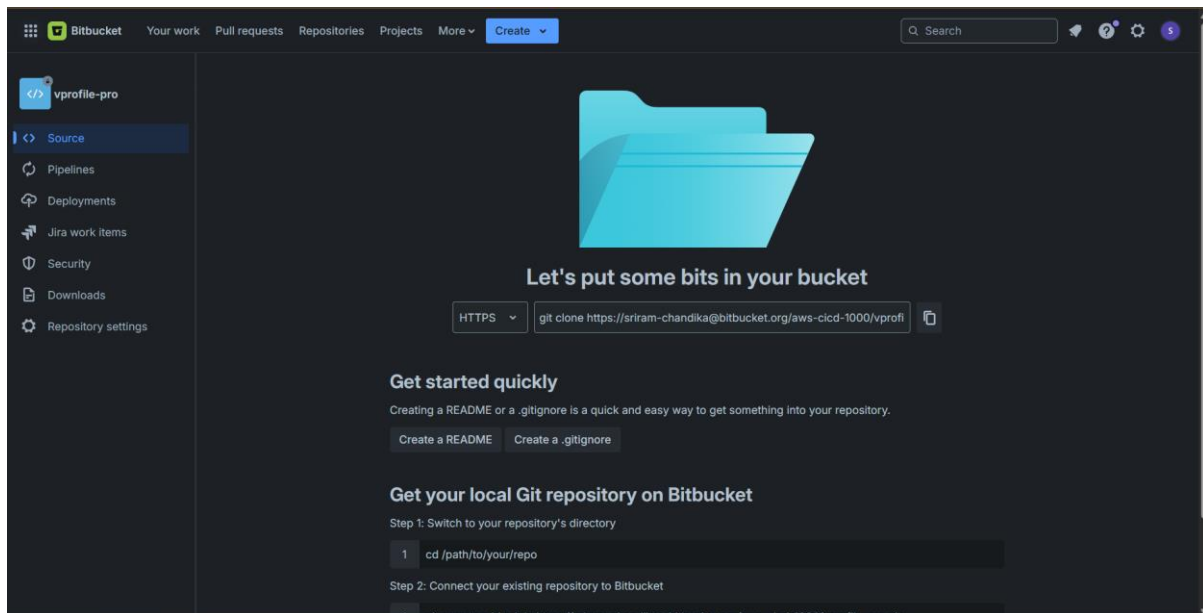
> Advanced settings

Create repository

Cancel

## 2. Create Repository

1. Inside the workspace, click **Create repository**.
2. Fill details:
  - Project Name: vprofile
  - Repository Name: vprofile-pro
  - Visibility: Private
  - Do **not** initialize with README or .gitignore (keep it empty).
3. Bitbucket will create the repo with a default **main** branch.



### 3. Generate SSH Keys

On your local machine:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~ (main)
$ cd ~/.ssh

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/.ssh (main)
$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/CHANDIKA SRIRAM/.ssh/id_ed
25519): bitbucket_rsa
Enter passphrase for "bitbucket_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in bitbucket_rsa
Your public key has been saved in bitbucket_rsa.pub
The key fingerprint is:
SHA256:2rycqxDbD6I8G00tHPSD0cHh22xICUCdbRmttqugzk CHANDIKA SRIRAM@CHANDIKA-SRIRAM
The key's randomart image is:
+--[ED25519 256]--+
|.OO.B.++O.|
|. .*=+O+|.
|+ =++ +|.
|BoB +|.
|.B.+ oS|
|.. .+|.
|O . ..O|
|E .. .O+|.
|... .O=O|
+-----[SHA256]-----+
```

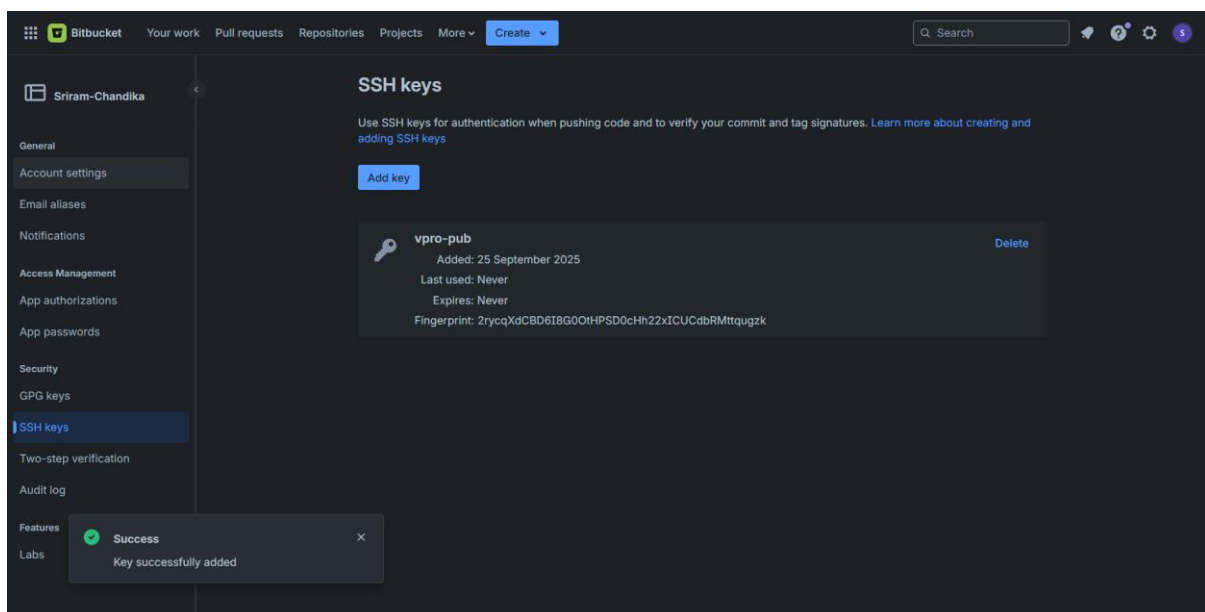
- When prompted, name it `bitbucket_rsa`.
- Press Enter for default location and skip passphrase.
- You will get two files:
  - `bitbucket_rsa` → private key
  - `bitbucket_rsa.pub` → public key

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/.ssh (main)
$ ls | grep bitbucket_rsa
bitbucket_rsa
bitbucket_rsa.pub
```

---

#### 4. Add SSH Key to Bitbucket

1. Copy public key.
2. In Bitbucket:
  - Go to **Personal Settings** → **SSH Keys** → **Add Key**
  - Paste the public key and save.



---

#### 5. Configure SSH Client

Create ~/.ssh/config file:

```
Host bitbucket.org
Host bitbucket.org
PreferredAuthentications publickey
IdentityFile ~/.ssh/bitbucket_rsa
```

This ensures Git uses your private key when pushing to Bitbucket.

---



## 6. Test Connection

Run:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~ (main)
$ ssh -T git@bitbucket.org
authenticated via ssh key.

You can use git to connect to Bitbucket. Shell access is disabled
```

- Say **yes** when asked to trust fingerprint.
- If successful, you'll see:  
*"You can use Git to connect to Bitbucket. Shell access is disabled."*

---

## 7. Verify Repository Clone (Optional Test)

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~ (main)
$ git clone git@bitbucket.org:aws-cicd-1000/vprofile-pro.git
Cloning into 'vprofile-pro'...
warning: You appear to have cloned an empty repository.
```

Since repo is empty, you'll see: *"You appear to have cloned an empty repository."*

---

## 8. Clone Vprofile Source Code from GitHub

```
MINGW64/c/Users/CHANDIKA SRIRAM/Desktop/bitbkt/vprofile-project

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~ (main)
$ mkdir Desktop/bitbkt

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~ (main)
$ cd Desktop/bitbkt

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt (main)
$ git clone https://github.com/sriramch163/vprofile-project.git
Cloning into 'vprofile-project'...
remote: Enumerating objects: 717, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 717 (delta 2), reused 2 (delta 2), pack-reused 714 (from 1)
Receiving objects: 100% (717/717), 184.48 MiB | 3.06 MiB/s, done.
Resolving deltas: 100% (53/53), done.

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt (main)
$ ls
vprofile-project/

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt (main)
$ cd vprofile-project
```

## 9. Checkout Required Branches

```
MINGW64~/c/Users/CHANDIKA SRIRAM/Desktop/bitbkt/vprofile-project
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (main)
$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (main)
$ git checkout aws-ci
branch 'aws-ci' set up to track 'origin/aws-ci'.
Switched to a new branch 'aws-ci'

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (aws-ci)
$ git fetch --tags
```

Only checked-out branches will be migrated.

---

## 10. Update Remote URL

1. Remove GitHub remote:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (aws-ci)
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote "origin"]
    url = https://github.com/sriramch163/vprofile-project.git
```

2. Add Bitbucket remote:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (aws-ci)
$ git remote rm origin

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (aws-ci)
$ git remote add origin git@bitbucket.org:aws-cicd-1000/vprofile-pro.git
```

3. Verify:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (aws-ci)
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote "origin"]
    url = git@bitbucket.org:aws-cicd-1000/vprofile-pro.git
    fetch = +refs/heads/*:refs/remotes/origin/*
```

→ Should show Bitbucket repo URL.

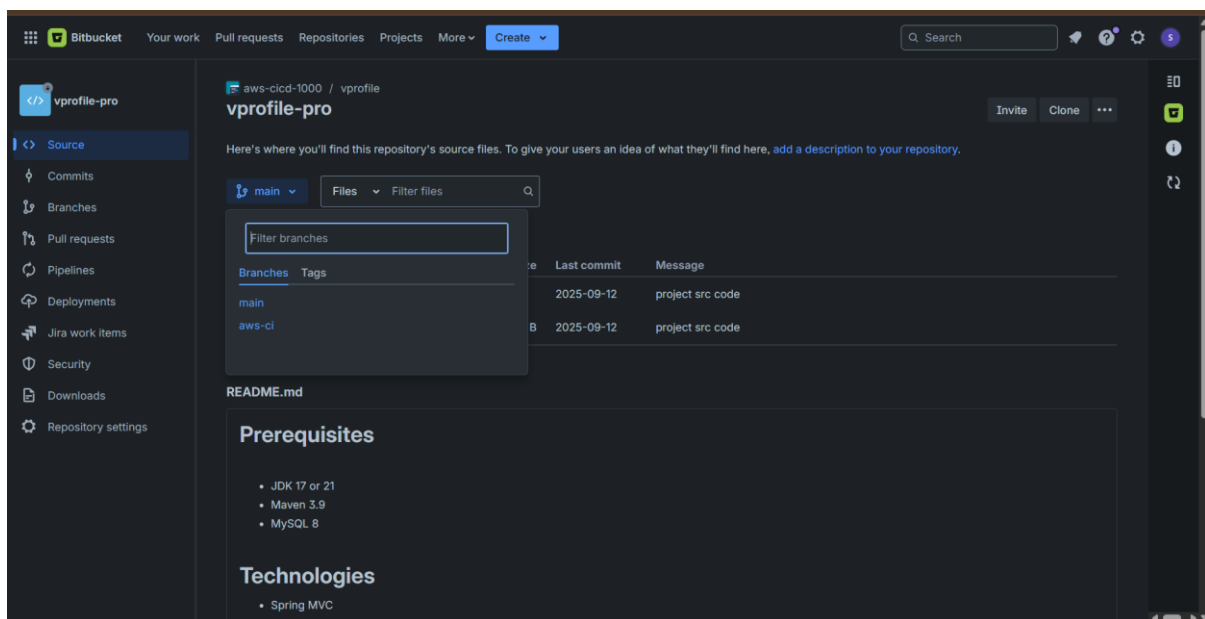
---

## 11. Push Code to Bitbucket

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/bitbkt/vprofile-project (aws-ci)
$ git push origin --all
Enumerating objects: 284, done.
Counting objects: 100% (284/284), done.
Delta compression using up to 8 threads
Compressing objects: 100% (239/239), done.
Writing objects: 100% (284/284), 9.89 MiB | 3.45 MiB/s, done.
Total 284 (delta 28), reused 280 (delta 27), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (28/28), done.
To bitbucket.org:aws-cicd-1000/vprofile-pro.git
 * [new branch]      aws-ci -> aws-ci
 * [new branch]      main -> main
```

## 12. Verify in Bitbucket

- Refresh repository in Bitbucket.
- You should see:
  - Source code migrated.
  - Branches `main` and `aws-ci`.



# 4.AWS CodeBuild Setup

## Objective

Use AWS CodeBuild to fetch source code from **Bitbucket**, compile it, update configuration (DB details), and generate an **artifact** stored in **S3** for deployment.

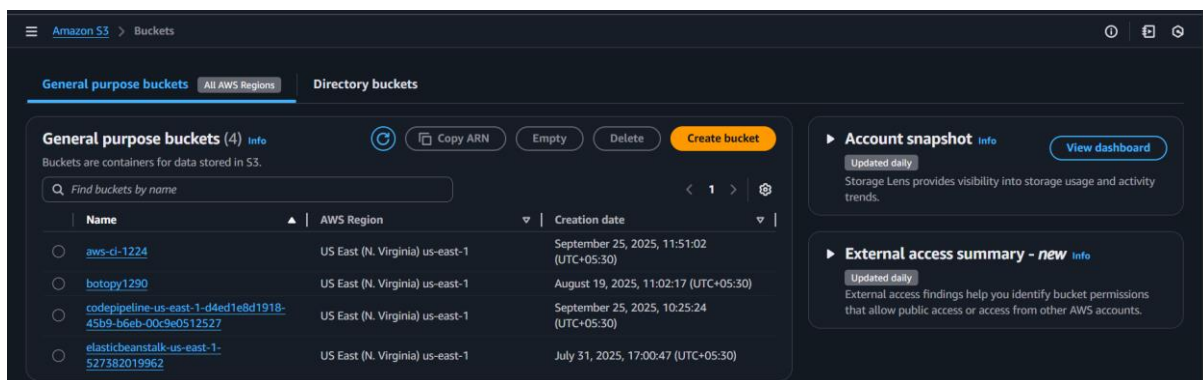
# Key Concepts

- **CodeBuild = Jenkins on AWS (managed service)**
    - No need for EC2 instances.
    - Pay only for build time.
  - **Artifacts** (build outputs) are stored in **Amazon S3**.
  - **Buildspec.yml**: YAML file that defines build steps, dependencies, and artifact packaging.
- 

## Steps

### 1. Create an S3 Bucket for Artifacts

1. Open **S3** service.
2. Create a bucket (must be unique). Example: `aws-ci-1224`
3. Keep it in the **same region** as CodeBuild.



### 2. Create CodeBuild Project

1. Open **CodeBuild** → **Create Project**.
2. **Project Name**: `vproappbuild`.
3. **Source Provider**: Select **Bitbucket**.
  - Create a **Bitbucket connection** (Bitbucket App).
  - Log in with your Bitbucket account (Google or username).
  - Select your workspace, repo (`vproapp`), and branch (`aws-ci`).

The screenshot shows the 'Create build project' page in the AWS CodeBuild console. The breadcrumb trail at the top reads: 'Developer Tools > CodeBuild > Build projects > Create build project'. The main heading is 'Create build project'. Below it is a 'Project configuration' section. The 'Project name' field contains 'vpro-app-aws-ci'. A note below the field states: 'A project name must be 2 to 255 characters. It can include the letters A-Z and a-z, the numbers 0-9, and the special characters - and \_.' The 'Project type' section has two radio buttons: 'Default project' (selected) and 'Runner project'. The 'Default project' description is 'Create a custom CodeBuild project.' The 'Runner project' description is 'Create a CodeBuild managed runner for workflows in GitHub Actions, GitHub Enterprise Actions, GitLab, or Buildkite.' At the bottom of the configuration section is an 'Additional configuration' link with a right-pointing arrow, with a tooltip that says 'Description, public build access, build badge, concurrent build limit, tags'.

Developer Tools > CodeBuild > Build projects > Create build project

## Create build project

### Project configuration

Project name

vpro-app-aws-ci

A project name must be 2 to 255 characters. It can include the letters A-Z and a-z, the numbers 0-9, and the special characters - and \_.

Project type

Select what type of project you would like to create. [Info](#)

☒ **Default project**  
Create a custom CodeBuild project.

☐ **Runner project**  
Create a CodeBuild managed runner for workflows in GitHub Actions, GitHub Enterprise Actions, GitLab, or Buildkite.

► **Additional configuration**  
Description, public build access, build badge, concurrent build limit, tags

### 3. Configure Build Environment

- **Provisioning Model:** On-demand (pay-as-you-go).
- **Environment Image:**
  - AWS managed image
  - OS: **Ubuntu**
  - Runtime: **Standard 7.0**
- **Service Role:** Create a new one.

The screenshot shows the 'Compute' configuration section of the AWS CodeBuild console. It has two radio buttons: 'EC2' (selected) and 'Lambda'. The 'EC2' description is 'Optimized for flexibility during action runs'. The 'Lambda' description is 'Optimized for speed and minimizes the start up time of workflow actions'. Below this is the 'Running mode' section with two radio buttons: 'Container' (selected) and 'Instance'. The 'Container' description is 'Running on Docker container'. The 'Instance' description is 'Running on EC2 instance directly'. Below these are several dropdown menus: 'Operating system' set to 'Ubuntu', 'Runtime(s)' set to 'Standard', 'Image' set to 'aws/codebuild/standard:7.0', and 'Image version' set to 'Always use the latest image for this runtime version'. At the bottom is a checkbox 'Use GPU-enhanced compute' which is unchecked, followed by an 'Additional configuration' link with a right-pointing arrow.

### Compute

☒ **EC2**  
Optimized for flexibility during action runs

☐ **Lambda**  
Optimized for speed and minimizes the start up time of workflow actions

### Running mode

☒ **Container**  
Running on Docker container

☐ **Instance**  
Running on EC2 instance directly

Operating system

Ubuntu

Runtime(s)

Standard

Image

aws/codebuild/standard:7.0

Image version

Always use the latest image for this runtime version

☐ Use GPU-enhanced compute

► **Additional configuration**

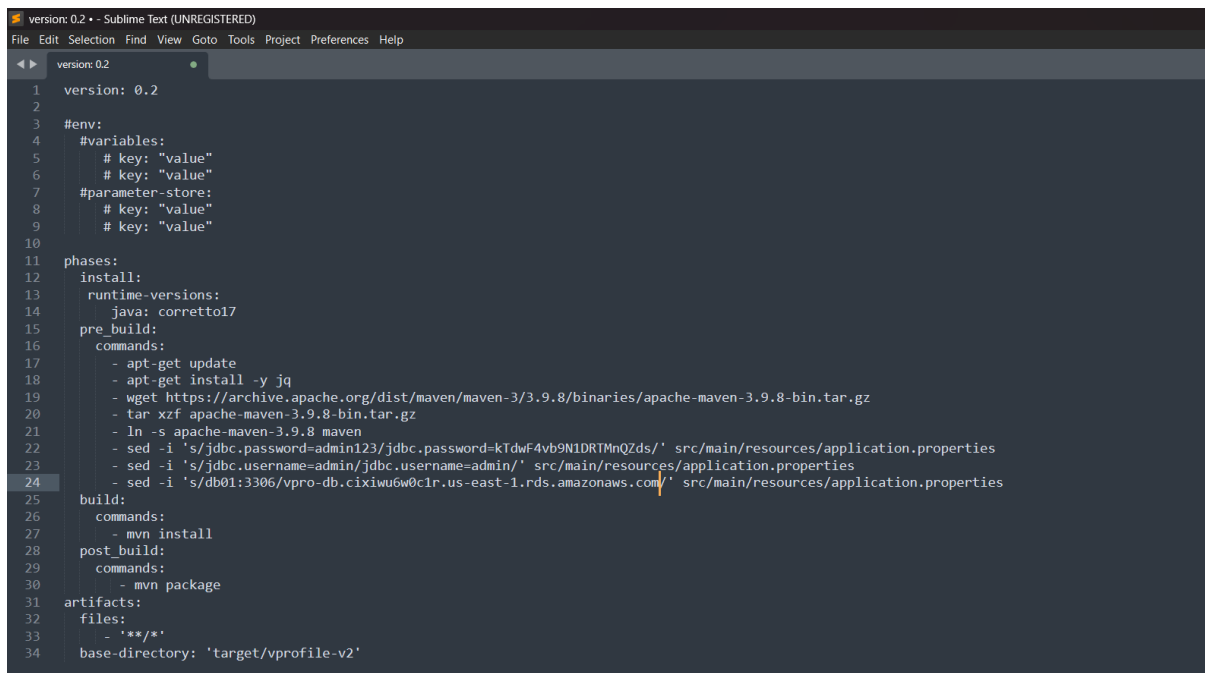
- **Compute Resources:** Minimum (3GB RAM, 2 vCPUs).

---

## 4. Buildspec File Setup

- Defines phases and commands.
- Can be stored in repo (`buildspec.yml`) or pasted directly.

**Example buildspec.yml** (simplified with DB configs):



```
version: 0.2
#env:
#variables:
# key: "value"
# key: "value"
#parameter-store:
# key: "value"
# key: "value"
phases:
  install:
    runtime-versions:
      java: corretto17
  pre_build:
    commands:
      - apt-get update
      - apt-get install -y jq
      - wget https://archive.apache.org/dist/maven/maven-3/3.9.8/binaries/apache-maven-3.9.8-bin.tar.gz
      - tar xzf apache-maven-3.9.8-bin.tar.gz
      - ln -s apache-maven-3.9.8 maven
      - sed -i 's/jdbc.password=admin123/jdbc.password=kTdwF4vb9N1DRTMnQzds/' src/main/resources/application.properties
      - sed -i 's/jdbc.username=admin/jdbc.username=admin/' src/main/resources/application.properties
      - sed -i 's/db01:3306/vpro-db.cixiwu6w0c1r.us-east-1.rds.amazonaws.com/' src/main/resources/application.properties
  build:
    commands:
      - mvn install
  post_build:
    commands:
      - mvn package
artifacts:
  files:
    - '**/*.*'
  base-directory: 'target/vprofile-v2'
```

🔑 Replace `<RDS_ENDPOINT>`, `<RDS_USERNAME>`, `<RDS_PASSWORD>` with actual **Amazon RDS** values.

---

## 5. Configure Artifacts

- **Type:** Amazon S3
- **Bucket:** `aws-ci-1224`

▼ Artifacts

Add artifact

Artifact 1 - Primary

Type

Amazon S3

You might choose no artifacts if you are running tests or pushing a Docker image to Amazon ECR.

Bucket name

Q aws-ci-1224

Name

The name of the folder or compressed file in the bucket that will contain your output artifacts. Use Artifacts packaging under Additional configuration to choose whether to use a folder or compressed file. If the name is not provided, defaults to project name.

☐ Enable semantic versioning

Use the artifact name specified in the buildspec file

Path - *optional*

The path to the build output ZIP file or folder.

Example: MyPath/MyArtifact.zip.

Namespace type - *optional*

None

- Path: leave empty (root).

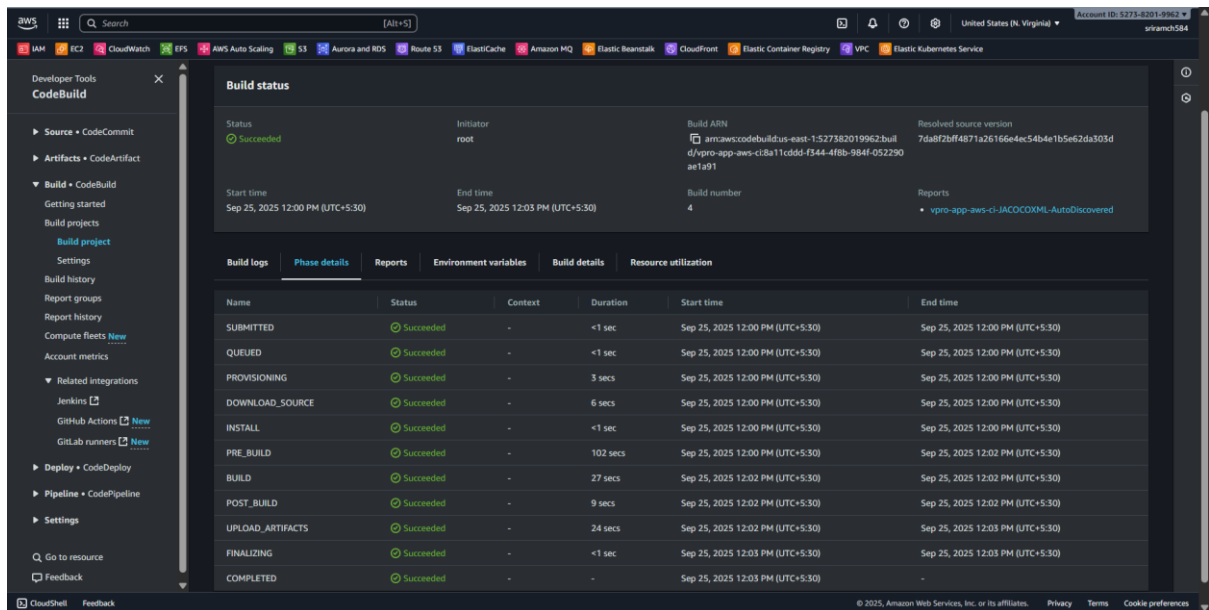
---

## 6. Enable Logging

- Enable **CloudWatch Logs**.
- Example:
  - Group: `vprofileproject`
  - Stream: `vprobuild`

# 5.Build, Deploy & CodePipeline

- **Test the CodeBuild project**
  - Run a manual build (`start build`) and debug logs if it fails.



- **Create CodePipeline**

- **Pipeline name:** vpro-app-aws-ci

## Step 2: Choose pipeline settings

### Pipeline settings

**Pipeline name**

vpro-app-aws-ci

**Pipeline type**

V2

**Execution mode**

QUEUED

**Artifact location**

codepipeline-us-east-1-d4ed1e8d1918-45b9-b6eb-00c9e0512527

**Service role name**

arn:aws:iam::527382019962:role/service-role/AWSCodePipelineServiceRole-us-east-1-vprofile-cicd

- **Source:** Bitbucket → repo (vproapp) → branch (aws-ci).



### Step 3: Add source stage

#### Source action provider

**Source action provider**

Bitbucket

**OutputArtifactFormat**

CODE\_ZIP

**DetectChanges**

true

**ConnectionArn**

arn:aws:codeconnections:us-east-1:527382019962:connection/4055c880-6b0f-40e1-a780-d7eb50e16cf9

**FullRepositoryId**

aws-cicd-1000/vprofile-pro

**Default branch**

aws-ci

**Enable automatic retry on stage failure**

Enabled

#### Trigger configuration

You can add additional pipeline triggers after the pipeline is created.

**Trigger type**

No filter

- **Build:** CodeBuild project (`vpro-app-aws-ci`).

### Step 4: Add build stage

#### Build action provider

**Build action provider**

AWS CodeBuild

**ProjectName**

vpro-app-aws-ci

**Commands**

-

**Enable automatic retry on stage failure**

Enabled

- **Deploy:** Elastic Beanstalk → select Application + Environment.

## Step 6: Add deploy stage

### Deploy action provider

**Deploy action provider**  
AWS Elastic Beanstalk

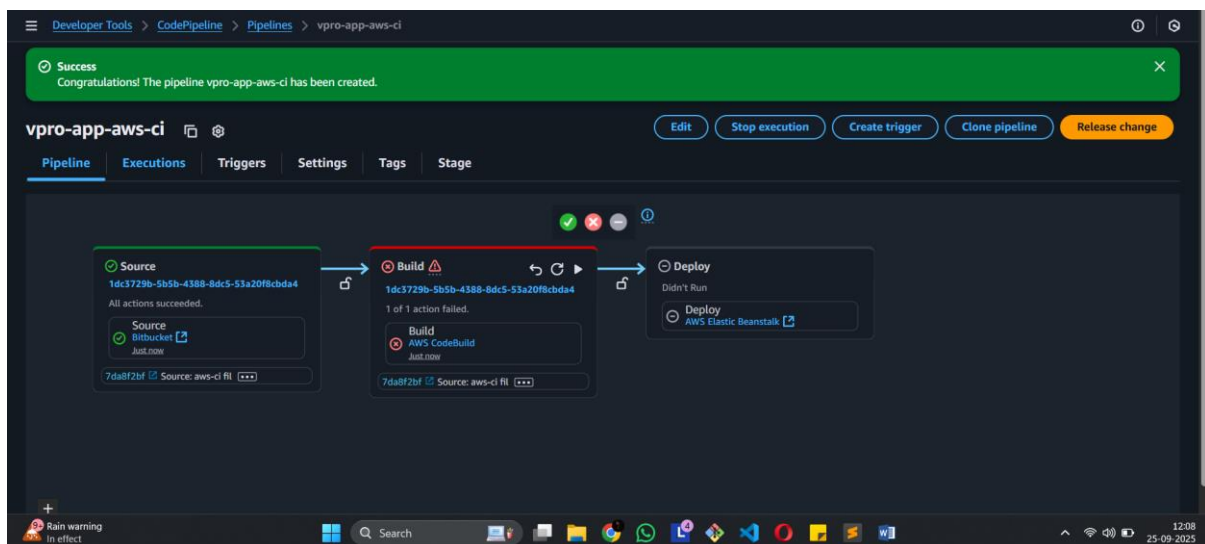
**ApplicationName**  
vprofile-bean-app

**EnvironmentName**  
Vprofile-bean-app-env

**Configure automatic rollback on stage failure**  
Enabled

**Enable automatic retry on stage failure**  
Disabled

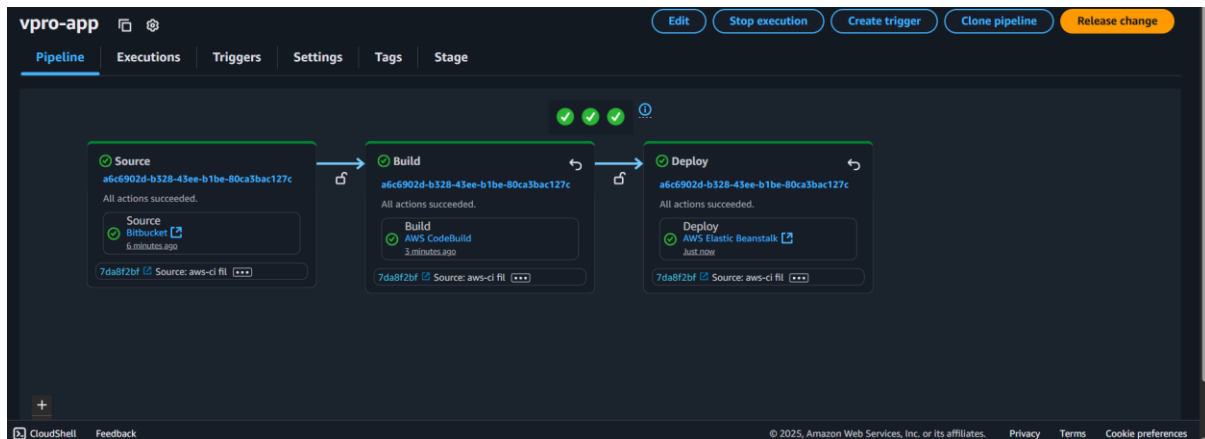
- AWS auto-creates a **service role**, but it **doesn't include Beanstalk permissions**.



- Fix: Add `AdministratorAccess-AWSElasticBeanstalk` policy to the IAM role.

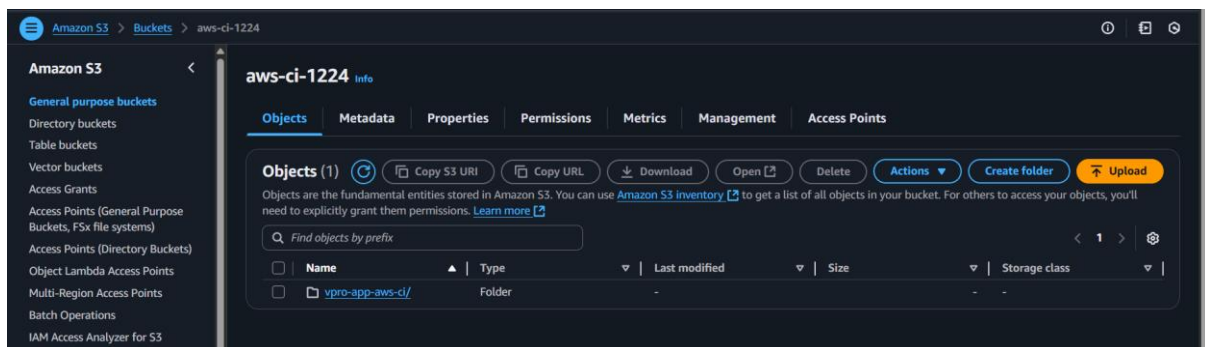
|                          |                                                              |                  |   |
|--------------------------|--------------------------------------------------------------|------------------|---|
| <input type="checkbox"/> | <a href="#">AdministratorAccess-AWSElasticBeanstalk</a>      | AWS managed      | 3 |
| <input type="checkbox"/> | <a href="#">AWSCodePipelineServiceRole-us-east-1-vp...</a>   | Customer managed | 1 |
| <input type="checkbox"/> | <a href="#">CodePipeline-CodeBuild-us-east-1-vprofile...</a> | Customer managed | 1 |
| <input type="checkbox"/> | <a href="#">CodePipeline-CodeConnections-us-east-1-...</a>   | Customer managed | 1 |

- Then when we runs the pipeline it runs successfully.



## • Trigger Pipeline

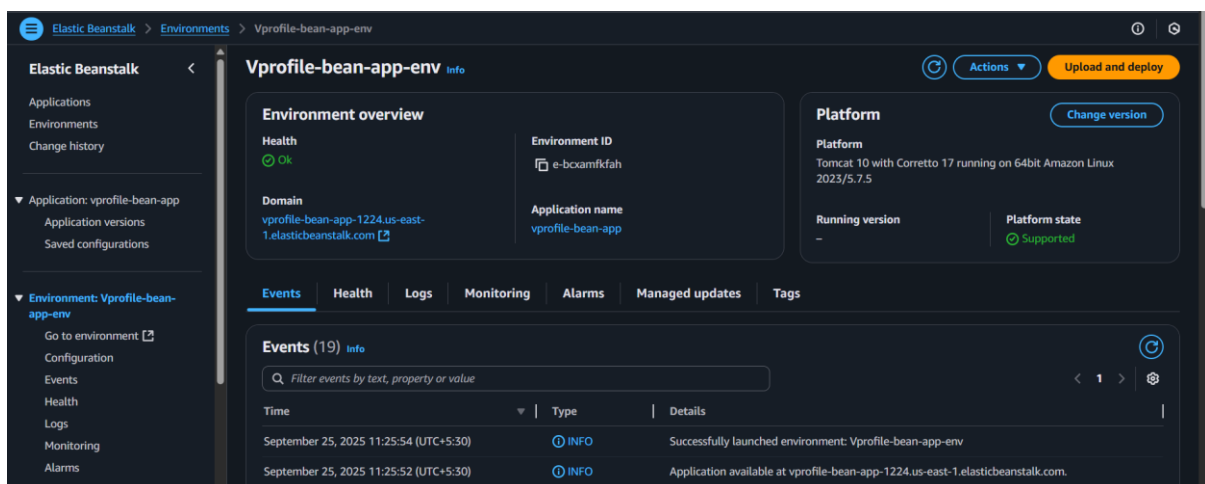
- CodePipeline detects commits to Bitbucket.
- Runs build (CodeBuild → artifact in S3).



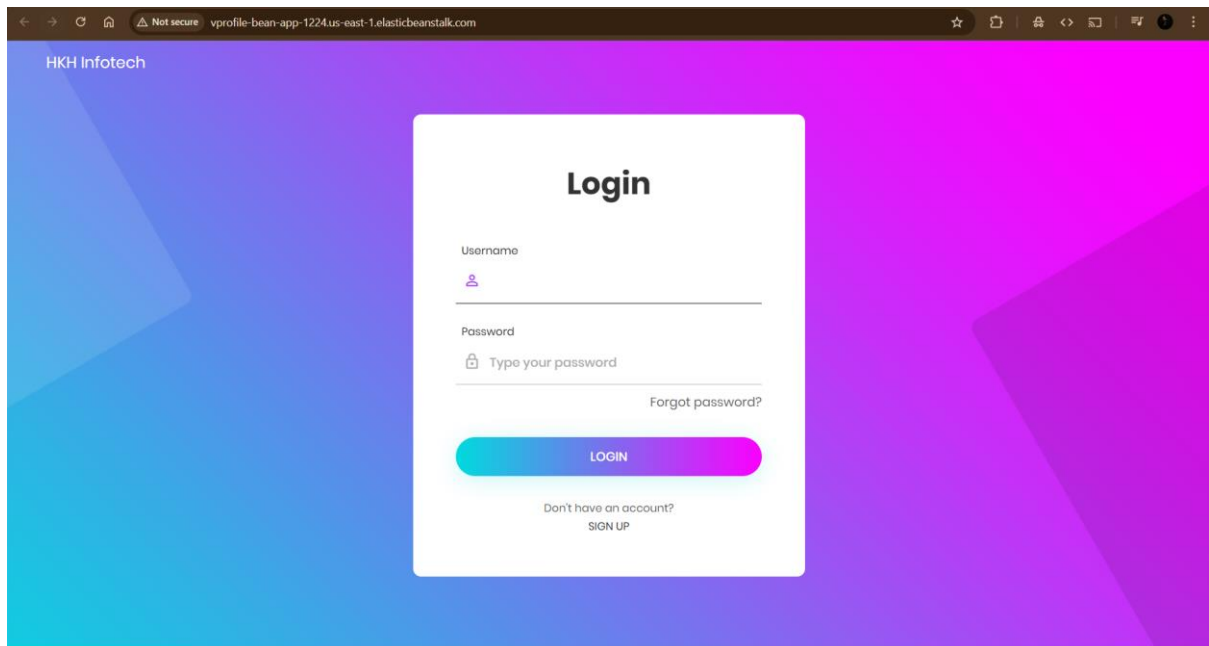
- Deploys to Elastic Beanstalk.
- Rolling deployment happens (1 instance updated at a time).

## • Validate Deployment

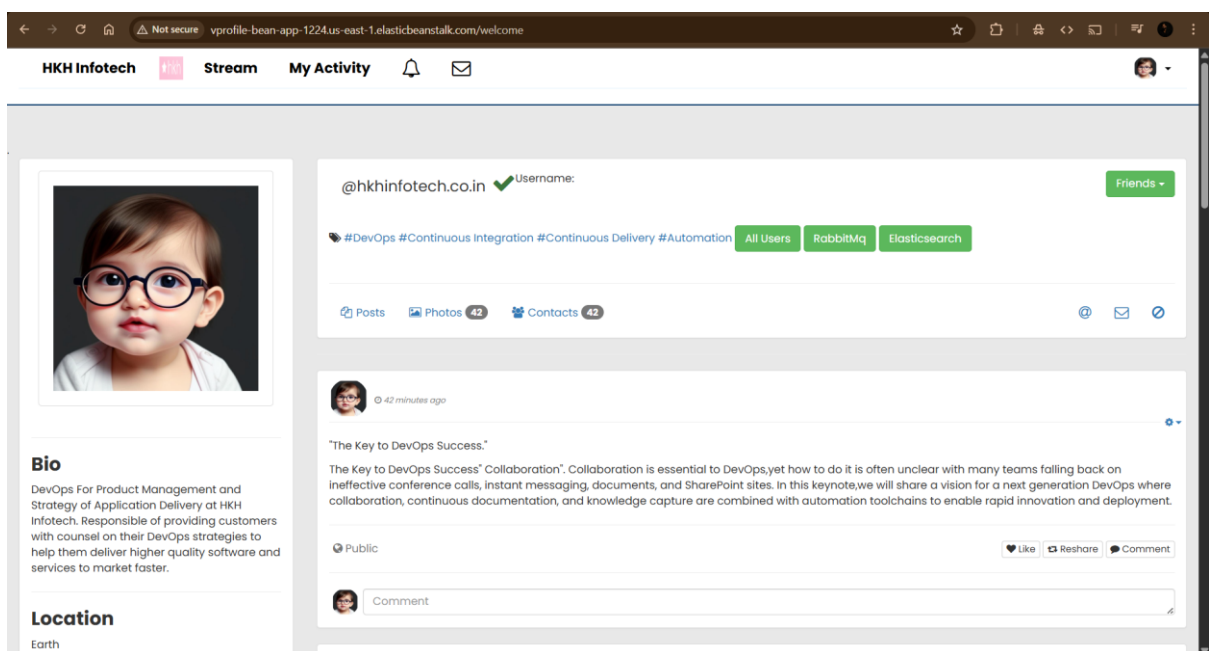
- Open Beanstalk environment URL (load balancer).



- Login with test credentials (admin\_vp / admin\_vp).

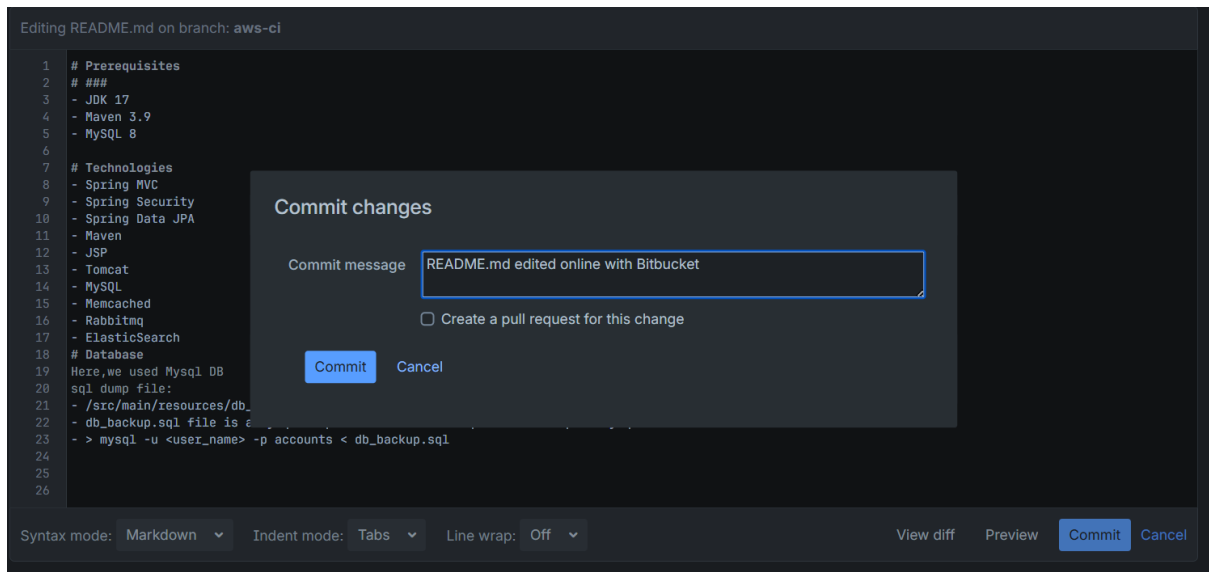


- Confirms app connects to **RDS database**.

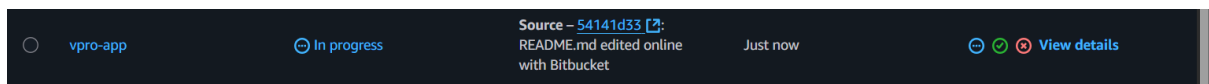


- **Automatic Trigger Test**

- Make a dummy commit in Bitbucket (README.md edit).



- Pipeline auto-triggers, rebuilds, redeploys.



- And next the pipeline successfull.

